Blockchain-Based NFT Digital Passport System – Full Comprehensive Documentation

This is the **complete expanded version** including: BRD, SRS, FRS, HLD, LLD, UML diagrams (text-based), DFDs, Database Schema, Smart Contract Pseudocode, Test Cases, Gantt Chart, Project Report Structure, and PPT outline.

---

1. BUSINESS REQUIREMENTS DOCUMENT (BRD)

1.1 Project Overview

Luxury brands lose billions due to counterfeit products. Consumers struggle to verify authenticity. Existing certificates (paper/QR) can be easily forged.

Solution: Each product receives a **Blockchain-based NFT Digital Passport** which acts as an immutable authenticity certificate. The digital passport is linked to a physical tag (QR/NFC/PUF).

When scanned, the system verifies metadata hash stored on blockchain and fetches original product metadata from off-chain storage.

1.2 Business Objectives

- Provide a decentralized, tamper-proof authenticity verification system.
- Enable consumers to instantly validate luxury products.
- Enable brands to track product lifecycle.
- Reduce counterfeit circulation.

1.3 In-Scope

- NFT digital passport issuance
- Blockchain metadata hashing
- QR/NFC scanning
- Verification engine
- Provenance timeline

1.4 Out-of-Scope

- Payment integration
- Marketplace features

• Advanced PUF hardware support (future)

1.5 Business Rules

• Each physical unit = One NFT Passport only.
• Metadata cannot be altered once minted.
• QR/NFC must uniquely map to NFT ID.

---

2. SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

2.1 Functional Requirements (Detailed)

FR1 – Product Registration

Admin must register a product with: - Serial number - Model name - Product images - Warranty documents

FR2 – Metadata Hashing

System must compute SHA-256 hash of metadata JSON.

FR3 – NFT Minting

Smart contract must mint ERC-721 token containing metadata hash.

FR4 – QR/NFC Binding

System must store mapping: `tag_id → token_id`.

FR5 – Verification Scan

When consumer scans tag: - Retrieve token_id - Fetch metadata hash from blockchain - Fetch off-chain metadata - Rehash and compare - Display Authentic / Fake / Tampered

FR6 – Provenance Logging

Supply chain staff can upload events: - Manufacturing - Quality check - Packaging - Shipping - Retail store arrival

FR7 – Product History Display

Consumer UI must show a timeline of all events.

2.2 Non-Functional Requirements (NFR)

Performance

- Verification time ≤ 2 seconds
- Minting time ≤ 5 seconds

Security

- Blockchain immutability
- Secure API keys
- No PII stored on-chain

Scalability

- Support 1M+ products
- IPFS redundancy

Reliability

- 99.99% uptime SLA

2.3 User Types

- **Brand Admin** (product registration + minting)
- **Supply Chain Staff** (event logging)
- **Consumer** (verification)
- **System Developer / QA**

---

3. FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

3.1 Module Descriptions (Expanded)

Module 1: Admin Panel

- Product form
- File upload
- NFT mint trigger
- Dashboard

Module 2: Verification Engine

- QR/NFC scan handler
- Metadata integrity check
- Blockchain resolver

Module 3: Blockchain Layer

- ERC-721 smart contract
- Metadata hashing
- Ownership logging

Module 4: Off-chain Storage

- IPFS or cloud bucket for large assets

Module 5: Consumer UI

- Scan interface
- Results page
- Product timeline

---

4. HIGH LEVEL DESIGN (HLD)

4.1 System Architecture Diagram (Text Version)

```
Physical Tag (QR/NFC/PUF)
        │
        ▼
Verification Gateway API → Blockchain Layer (NFT Passport)
        │                          │
        ▼                          ▼
   Off-chain Storage ← Metadata Hash Validation
        │
        ▼
   Consumer UI (Verification Result)
```

4.2 Data Flow Diagram (DFD)

Level 0 DFD

```
Consumer → Scan → Verification API → Blockchain + Storage → Result
```

Level 1 DFD

```
Scan Tag
   ↓
Backend resolves tokenID
   ↓
Fetch blockchain hash
   ↓
Fetch metadata JSON
   ↓
Recompute hash
   ↓
Compare
   ↓
Show authenticity result
```

---

5. LOW LEVEL DESIGN (LLD)

5.1 Database Schema (Detailed)

Table: products

Field	Type	Description
product_id	UUID	Primary key
name	varchar	Product name
serial_number	varchar	Unique identifier
created_at	timestamp	Timestamp

Table: nft_records

Field	Type
token_id	int
product_id	UUID
metadata_hash	varchar
transaction_hash	varchar

Table: verification_logs

| scan_id | UUID | | tag_id | varchar | | status | varchar (Authentic/Fake) | | timestamp | timestamp |

5.2 API Contract (Backend)

POST /api/admin/product

Request: JSON Response: product_id

POST /api/admin/mint-nft

Request: product_id Response: token_id

GET /api/verify/{tag_id}

Response:

```
{
  "status": "Authentic",
  "product": {...},
  "timeline": [...]
}
```

---

6. SMART CONTRACT SPECIFICATION

6.1 Pseudocode

```
contract NFTPassport is ERC721 {
    mapping(uint256 => string) public metadataHash;

    function mintPassport(address owner, string memory hash) public
returns(uint256) {
        uint256 tokenId = totalSupply + 1;
        _mint(owner, tokenId);
        metadataHash[tokenId] = hash;
        emit PassportMinted(tokenId, owner, hash);
        return tokenId;
    }

    function getHash(uint256 tokenId) public view returns(string memory) {
```

```
        return metadataHash[tokenId];
    }
}
```

---

7. FRONTEND DEVELOPER TASKS

Admin Panel

- React dashboard
- Product form UI
- Metadata upload
- “Mint NFT” button connected to API

Consumer App

- QR scanner integration
- Responsive UI
- Authentication result page
- Provenance timeline UI

---

8. BACKEND DEVELOPER TASKS

Core Responsibilities

- Build all REST APIs
- Metadata hashing module
- Blockchain connector
- QR/NFC resolver service
- IPFS integration
- DB ORM setup
- Authentication middleware

---

9. TEST CASE DOCUMENT

Test ID	Case	Input	Expected Output
TC01	Mint NFT	Valid metadata	NFT ID generated
TC02	Scan	Valid QR	Authentic

Test ID	Case	Input	Expected Output
TC03	Scan	Tampered metadata	Tampered
TC04	Scan	Fake QR	Fake

---

10. GANTT CHART (Text Version)

```
Week 1–2: Requirements + Architecture
Week 3–4: Smart Contract
Week 5–6: Backend API
Week 7–8: Frontend Development
Week 9: Integration
Week 10: Testing + Documentation
```

---

11. FINAL YEAR PROJECT REPORT STRUCTURE

- Abstract
- Introduction
- Literature Survey
- System Analysis
- Methodology
- Architecture Diagram
- Module Description
- Implementation
- Results
- Conclusion
- Future Work

---

12. PPT OUTLINE

- Title Slide
- Problem Statement
- Proposed Solution
- Architecture
- Workflow
- Demo Screens
- Smart Contract
- Results
- Future Scope

• Q/A